

EVIDENCIAS DE PRUEBAS – MARÍA ALEJANDRA FONSECA ZARRATE

```
1 ##AP-SERVIDOR
2 import network, time, socket
3
4 # Configuración del AP
5 ap = network.WLAN(network.AP_IF)
6 ap.config(essid='WLAN_IoT_UMNG', password='SeguridadUMNG2024!', security=3)
7 ap.config(channel=6, txpower=20) # Parámetros avanzados
8 ap.active(True)
9
10 print("--- SERVIDOR AP INICIADO ---")
11 print("Nombre de red:", 'WLAN_IoT_UMNG')
12 print("IP del Servidor:", ap.ifconfig()[0])
13
14 # Abrir el puerto para recibir datos (UDP)
15 s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
16 s.bind(('', 8080))
17 print("Esperando datos en el puerto 8080...")
18
19 while True:
20     data, addr = s.recvfrom(1024)
21     print(f"RECIBIDO de {addr}: {data.decode('utf-8')}")
```

Consola

```
>>> %Run -c $EDITOR_CONTENT
```

```
MPY: soft reboot
--- SERVIDOR AP INICIADO ---
Nombre de red: WLAN_IoT_UMNG
IP del Servidor: 192.168.4.1
Esperando datos en el puerto 8080...
```

The screenshot displays two instances of the Thonny IDE. The left instance shows a Python script for an AP server, and the right instance shows a Python script for a client that connects to the AP and sends sensor data.

Left Window (AP Server Script):

```
30 intentos += 1
31
32 if ap.active():
33     print("--- RED LISTA ---")
34     print("Nombre (SSID): WLAN_IoT_UMNG")
35     print("IP del Servidor:", ap.ifconfig()[0])
36     # Encender LED interno para confirmar
37     machine.Pin("LED", machine.Pin.OUT).on()
38 else:
39     print("Fallo crítico: La antena no enciende.")
40
41 iniciar_ap()
42
43 # 3. Mantener el programa vivo para que la red no desaparez
44 import socket
45 s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
46 s.bind(('', 8080))
47 print("Escaneando con el celular ahora mismo...")
48
49 while True:
50     try:
51         data, addr = s.recvfrom(1024)
52         print(f"Recibido de {addr}: {data.decode()}")
```

Right Window (Client Script):

```
21 while True:
22     # 1. Medir RSSI (Potencia de señal)
23     rssi = wlan.status('rssi')
24
25     # 2. Medir Temperatura interna
26     lectura = sensor_temp.read_u16() * (3.3 / 65535)
27     temp = 27 - (lectura - 0.706) / 0.001721
28
29     # Crear mensaje
30     payload = f"Temp: {temp:.2f}C | RSSI: {rssi}dBm"
31
32     # Enviar al AP
33     try:
34         s.sendto(payload.encode('utf-8'), (server_ip, 8080))
35         print("Enviado:", payload)
36
37     except:
38         # Parpadeo LED externo al enviar
39         led_extern.on()
40         time.sleep(0.1)
41         led_extern.off()
42     print("Error de envío")
```

Console Output (Left Window):

```
>>> %Run -c $EDITOR_CONTENT
MPY: soft reboot
Desactivando antena para limpieza...
Activando antena...
Configuración aplicada con éxito.
--- RED LISTA ---
Nombre (SSID): WLAN_IoT_UMNG
IP del Servidor: 192.168.4.1
Escaneando con el celular ahora mismo...
```

Console Output (Right Window):

```
>>> %Run -c $EDITOR_CONTENT
MPY: soft reboot
Conectando al AP...
¡Conectado! IP: 192.168.4.16
Enviado: Temp: 27.04C | RSSI: -44dBm
Enviado: Temp: 29.05C | RSSI: -45dBm
Enviado: Temp: 30.32C | RSSI: -45dBm
Enviado: Temp: 30.32C | RSSI: -43dBm
Enviado: Temp: 30.32C | RSSI: -42dBm
Enviado: Temp: 30.32C | RSSI: -42dBm
```

1METRO

```
[ main.py ] ×
1  #CLIENTE
2  import network
3  import socket
4  import time
5  import machine
6
7  # Configuración de Hardware
8  led_externo = machine.Pin(15, machine.Pin.OUT)
9  sensor_temp = machine.ADC(4)
10
11 # 1. Preparar la antena del Cliente
12 wlan = network.WLAN(network.STA_IF)
13 wlan.active(False)
14 time.sleep(1)
15 wlan.active(True)
16
17 # Credenciales (REVISA QUE SEAN EXACTAMENTE IGUALES AL AP)
18 SSID_RED = 'WLAN_IoT_UMNG'
19 PASS_RED = 'SeguridadUMNG2024!'
20
21 print(f"\nBuscando red: {SSID_RED}...")
22
23 # 2. Intentar conexión con reintentos
24 wlan.connect(SSID_RED, PASS_RED)
```

Consola ×

```
Enviado al Servidor: Temp: 33.6C | RSSI: -63dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -65dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -64dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -56dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -53dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -68dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -75dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -73dBm
Enviado al Servidor: Temp: 34.1C | RSSI: -55dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -60dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -59dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -59dBm
```

5 METROS

```
[ main.py ] ×
1  #CLIENTE
2  import network
3  import socket
4  import time
5  import machine
6
7  # Configuración de Hardware
8  led_externo = machine.Pin(15, machine.Pin.OUT)
9  sensor_temp = machine.ADC(4)
10
11 # 1. Preparar la antena del Cliente
12 wlan = network.WLAN(network.STA_IF)
13 wlan.active(False)
14 time.sleep(1)
15 wlan.active(True)
16
17 # Credenciales (REVISA QUE SEAN EXACTAMENTE IGUALES AL AP)
18 SSID_RED = 'WLAN_IoT_UMNG'
19 PASS_RED = 'SeguridadUMNG2024!'
20
21 print(f"\nBuscando red: {SSID_RED}...")
22
23 # 2. Intentar conexión con reintentos
24 wlan.connect(SSID_RED, PASS_RED)
```

```
Consola ×
Enviado al Servidor: Temp: 33.1C | RSSI: -79dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -80dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -85dBm
Enviado al Servidor: Temp: 32.7C | RSSI: -86dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -83dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -82dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -86dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -84dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -80dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -81dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -82dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -80dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -80dBm
```

10 METROS

```
[ main.py ] x
1 #CLIENTE
2 import network
3 import socket
4 import time
5 import machine
6
7 # Configuración de Hardware
8 led_externo = machine.Pin(15, machine.Pin.OUT)
9 sensor_temp = machine.ADC(4)
10
11 # 1. Preparar la antena del Cliente
12 wlan = network.WLAN(network.STA_IF)
13 wlan.active(False)
14 time.sleep(1)
15 wlan.active(True)
16
17 # Credenciales (REVISA QUE SEAN EXACTAMENTE IGUALES AL AP)
18 SSID_RED = 'WLAN_IoT_UMNG'
19 PASS_RED = 'SeguridadUMNG2024!'
20
21 print(f"\nBuscando red: {SSID_RED}...")
22
23 # 2. Intentar conexión con reintentos
24 wlan.connect(SSID_RED, PASS_RED)
```

```
Consola x
Enviado al Servidor: Temp: 33.1C | RSSI: -92abm
Enviado al Servidor: Temp: 33.1C | RSSI: -92dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -91dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -95dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -95dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -92dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -89dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -91dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -91dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -91dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -90dBm
Enviado al Servidor: Temp: 33.1C | RSSI: -90dBm
Enviado al Servidor: Temp: 33.6C | RSSI: -91dBm
```

LATENCIA

Ip asignada por ap

```
adaptador de LAN inalámbrica Wi-Fi:

Sufijo DNS específico para la conexión. . . :
Vínculo: dirección IPv6 local. . . : fe80::bfb7:735d:ccae:ad34%21
Dirección IPv4. . . . . : 192.168.4.17
Máscara de subred . . . . . : 255.255.255.0
Puerta de enlace predeterminada . . . . . : 192.168.4.1
```

Ping al servidor

```

C:\Users\User>ping 192.168.4.1

Haciendo ping a 192.168.4.1 con 32 bytes de datos:
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255
Respuesta desde 192.168.4.1: bytes=32 tiempo=2ms TTL=255

Estadísticas de ping para 192.168.4.1:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 2ms, Máximo = 2ms, Media = 2ms

C:\Users\User>

```

